ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN

BÀI TẬP NHÓM

XẤP XỈ HÀM GAUSSIAN 2D BẰNG MẠNG NƠ-RON NHÂN TẠO TRONG PYTORCH

Học phần:	Học sâu (Deep Learning)
Mã học phần:	858048
Nhóm lớp:	01
Giảng viên hướng dẫn:	**TS. Đỗ Như Tài**
Sinh viên thực hiện:	Đặng Thanh Phương: 3123580036 Phạm Trung Hiếu: 3123580014 Nguyễn Trương Cao Sơn: 3123580040 (Nhóm trưởng)
Lớp:	DDU1231
Học kỳ / Năm học:	Học kỳ 1, năm học 2026–2027

Thành phố Hồ Chí Minh, tháng 10 năm 2026

BẢNG PHÂN CÔNG NHIỆM VỤ VÀ SẢN PHẨM ĐẦU RA

Thành viên	Vai trò và Nhiệm vụ chính	Đầu ra	% Đóng góp
Đặng Thanh Phương (3123580036)	Vai trò: Quản lý, Trực quan hóa 3D & Thực nghiệm C - Quản lý Git, điều phối dự án. - Code Trực quan hóa 3D (Ground Truth, mô hình, sai số). - Khảo sát độ sâu mạng (C ∈ [1..6]), phân tích Vanishing Gradient. - Tổng hợp Báo cáo và làm Slide.	- visualize.py, exp_C_depth.py - Log/Ảnh 3D phần C - File Báo cáo (Word/PDF) - Slide (PPTX)	34%
Nguyễn Trương Cao Sơn (Nhóm trưởng) (3123580040)	Vai trò: Kiến trúc MLP & Thực nghiệm B - Xây dựng MLPRegressor, Trainer, DataLoader (PyTorch). - Xây dựng Scaler và các hàm Metrics. - Khảo sát độ rộng tầng ẩn (B ∈ [1..64]). - Viết báo cáo phần Kiến trúc & Thực nghiệm B.	- model.py, trainer.py, config.py... - exp_B_width.py - best_model.pt - Log/Biểu đồ phần B	33%
Phạm Trung Hiếu (3123580014)	Vai trò: Pipeline Dữ liệu & Thực nghiệm A - Sinh và chia dữ liệu (Gaussian 2D) bằng PCG64. - Khảo sát quy mô tập Train (A ∈ [200..2000]). - Phân tích Learning Curve. - Viết báo cáo phần Dữ liệu & Thực nghiệm A.	- data.py - train_full, val, test.csv - exp_A_samples.py - Log/Biểu đồ phần A	33%

LỜI MỞ ĐẦU

Trong học sâu, mạng nơ-ron truyền thẳng nhiều tầng (MLP) là một kiến trúc cơ bản, thường được sử dụng cho các bài toán phân loại và hồi quy. Về mặt lý thuyết, Cybenko (1989) chỉ ra rằng mạng có một tầng ẩn với hàm kích hoạt sigmoid liên tục phù hợp có thể xấp xỉ đều các hàm liên tục trên một miền compact. Những kết quả sau đó của Hornik (1991) mở rộng điều kiện đối với hàm kích hoạt và làm rõ thêm khả năng xấp xỉ của mạng nơ-ron.

Tuy nhiên, định lý xấp xỉ phổ quát chủ yếu nói về khả năng biểu diễn của mô hình khi có đủ nơ-ron; định lý không trực tiếp cho biết cần bao nhiêu mẫu dữ liệu, độ rộng hoặc độ sâu nào sẽ phù hợp với một bài toán cụ thể. Kết quả thực tế còn phụ thuộc vào cách sinh dữ liệu, khởi tạo trọng số, thuật toán tối ưu và cấu hình huấn luyện.

Từ yêu cầu của học phần Học sâu tại Trường Đại học Sài Gòn, nhóm thực hiện đề tài "Xấp xỉ hàm mật độ xác suất Gaussian 2 chiều bằng mạng nơ-ron MLP trong PyTorch". Hàm mục tiêu được chọn là phân phối chuẩn hai biến có phương sai theo hai trục khác nhau và có hiệp phương sai âm. Đặc điểm này tạo ra các đường đồng mức dạng elip nghiêng, phù hợp để khảo sát khả năng biểu diễn của mạng trong không gian hai chiều.

Báo cáo tập trung vào ba nội dung thực nghiệm và phần trực quan hóa kết quả:
1. Thực nghiệm A: khảo sát ảnh hưởng của số lượng mẫu huấn luyện từ 200 đến 2.000 mẫu.
2. Thực nghiệm B: khảo sát ảnh hưởng của số nơ-ron trong một tầng ẩn, với độ rộng từ 1 đến 64.
3. Thực nghiệm C: khảo sát ảnh hưởng của số tầng ẩn từ 1 đến 6, giữ cố định 32 nơ-ron ở mỗi tầng.
4. Trực quan hóa: so sánh mặt cong lý thuyết với mặt cong do mô hình dự đoán và xem xét phân bố sai số trên miền đầu vào.

Báo cáo gồm bảy chương, lần lượt trình bày bài toán và cơ sở lý thuyết, cách tạo và kiểm tra dữ liệu, kiến trúc mô hình, ba thực nghiệm, kết quả trực quan hóa và phần kết luận. Nhóm mong nhận được góp ý của Thầy TS. Đỗ Như Tài để tiếp tục hoàn thiện bài làm.

Nhóm chúng em xin chân thành cảm ơn!

LỜI CẢM ƠN

Nhóm sinh viên xin chân thành cảm ơn Thầy TS. Đỗ Như Tài, giảng viên phụ trách học phần Học sâu tại Trường Đại học Sài Gòn, đã hướng dẫn và góp ý trong quá trình thực hiện bài tập lớn.

Trong học phần, Thầy đã giúp nhóm tiếp cận các kiến thức về mạng nơ-ron nhân tạo, lan truyền thuận, lan truyền ngược, hàm mất mát và phương pháp tối ưu. Những nội dung này là cơ sở để nhóm xây dựng mô hình, xử lý dữ liệu và tổ chức các lần chạy thực nghiệm bằng PyTorch.

Thông qua đề tài, các thành viên có dịp thực hành từ bước xây dựng hàm mục tiêu, sinh và chia dữ liệu, chuẩn hóa đặc trưng đến huấn luyện, đánh giá và trực quan hóa kết quả. Quá trình làm việc cũng giúp nhóm nhận ra những điểm cần kiểm tra kỹ hơn khi diễn giải số liệu và so sánh các cấu hình mô hình.

Nhóm xin cảm ơn Thầy vì sự hướng dẫn trong suốt học phần. Kính chúc Thầy sức khỏe và thành công trong công tác giảng dạy, nghiên cứu.

Tập thể sinh viên thực hiện nhóm 01:
Đặng Thanh Phương
Phạm Trung Hiếu
Nguyễn Trương Cao Sơn (Nhóm trưởng)

MỤC LỤC

1. HIỂU BÀI TOÁN & CƠ SỞ LÝ THUYẾT 8
1.1. Phát biểu bài toán xấp xỉ hàm mật độ xác suất Gaussian 2D 8
1.2. Tính toán giải tích và kiểm định ma trận 8
1.3. Đặc điểm hình học của phân phối 8
1.4. Định lý Xấp xỉ Phổ quát 8
1.5. Trực quan hóa mặt cong lý thuyết trong không gian 3D 9
2. THIẾT KẾ VÀ KIỂM ĐỊNH BỘ DỮ LIỆU 10
2.1. Miền xác định và cách sinh dữ liệu 10
2.2. Kiểm tra tính độc lập và toàn vẹn của dữ liệu 10
2.3. Trực quan hóa phân bố mẫu huấn luyện 10
2.4. Chuẩn hóa đặc trưng và kiểm soát rò rỉ dữ liệu 11
3. XÂY DỰNG MÔ HÌNH MLP VÀ QUY TRÌNH HUẤN LUYỆN 11
3.1. Thiết kế kiến trúc mạng MLPRegressor trong PyTorch 11
3.2. Lựa chọn hàm kích hoạt Tanh 12
3.3. Tầng đầu ra tuyến tính và hàm mất mát MSE 12
3.4. Thuật toán tối ưu Adam và cấu hình huấn luyện cơ sở 12
3.5. Huấn luyện theo mini-batch và lưu mô hình tốt nhất 13
4. THỰC NGHIỆM A — ẢNH HƯỞNG CỦA QUY MÔ DỮ LIỆU HUẤN LUYỆN 13
4.1. Thiết kế thực nghiệm và cách đánh giá 13
4.2. Kết quả thực nghiệm A 13
4.3. Đường cong sai số theo quy mô dữ liệu 14
4.4. Nhận xét về ảnh hưởng của số lượng mẫu 14
5. THỰC NGHIỆM B — ẢNH HƯỞNG CỦA ĐỘ RỘNG TẦNG ẨN 15
5.1. Thiết kế thực nghiệm với một tầng ẩn 15
5.2. Kết quả thực nghiệm B 15
5.3. Nhận xét về độ rộng tầng ẩn 16
5.4. Số tham số và thời gian huấn luyện 17
6. THỰC NGHIỆM C — ẢNH HƯỞNG CỦA ĐỘ SÂU MẠNG VÀ TRỰC QUAN HÓA 3D 17
6.1. Thiết kế thực nghiệm với nhiều tầng ẩn 17
6.2. Kết quả theo độ sâu mạng 18

6.3. Quá trình thay đổi hàm mất mát 18
6.4. So sánh kết quả theo độ sâu và bàn luận về tiêu biến gradient 19
6.5. So sánh mặt cong dự đoán với mặt cong lý thuyết 19
6.6. Phân tích bề mặt sai số 20
7. TỔNG KẾT, THẢO LUẬN VÀ KẾT LUẬN 21
7.1. Tổng hợp ba trục thực nghiệm A, B và C 21
7.2. Cấu hình đề xuất cho bài toán 21
7.3. Khả năng tái lập và hướng dẫn chạy mã nguồn 21
7.4. Kết luận và kinh nghiệm thực hiện 22
7.5. Tài liệu tham khảo và thông tin bàn giao 22

DANH MỤC HÌNH VẼ & BẢNG BIỂU

I. DANH MỤC BẢNG BIỂU

Bảng 2.1: Quy mô và cách phân chia các tập dữ liệu thực nghiệm
Bảng 3.1: Cấu hình huấn luyện cơ sở
Bảng 4.1: Kết quả thực nghiệm A theo số lượng mẫu huấn luyện
Bảng 5.1: Kết quả thực nghiệm B theo độ rộng tầng ẩn
Bảng 6.1: Kết quả thực nghiệm C theo số tầng ẩn
Bảng 7.1: Tổng hợp kết quả ba thực nghiệm A, B và C

II. DANH MỤC HÌNH VẼ

Hình 1.1: Mặt cong lý thuyết của hàm mật độ Gaussian 2D
Hình 2.1: Phân bố 3D của 500 điểm hiển thị từ tập train_full gồm 2.000 mẫu
Hình 3.1: Sơ đồ kiến trúc tổng quát của mạng MLPRegressor
Hình 4.1: Đường cong sai số theo số lượng mẫu huấn luyện
Hình 5.1: Sai số theo độ rộng tầng ẩn
Hình 5.2: Đối chiếu giá trị thực và giá trị dự đoán với B = 64
Hình 6.1: Sai số kiểm thử và thời gian huấn luyện theo độ sâu mạng
Hình 6.2: Đường cong hàm mất mát theo epoch
Hình 6.3: Minh họa mặt cong dự đoán của mô hình C = 4 theo nhãn trên hình
Hình 6.4: Bề mặt sai số tuyệt đối |y − ŷ| trên miền khảo sát

1. HIỂU BÀI TOÁN & CƠ SỞ LÝ THUYẾT

1.1. Phát biểu bài toán xấp xỉ hàm mật độ xác suất Gaussian 2D

Bài toán được đặt dưới dạng hồi quy: đầu vào là tọa độ hai chiều (x_1, x_2), đầu ra là giá trị mật độ xác suất y của phân phối chuẩn hai biến. Mạng MLP được huấn luyện để dự đoán giá trị này trên miền khảo sát đã xác định.

> **Công thức mật độ xác suất Gaussian hai biến**
>
> $$y = f(x_1, x_2) = 1 / (2\pi\sqrt{\det(\Sigma)}) \cdot \exp[-\tfrac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)]$$
>
> Với $\Sigma = [[0.3, -0.5], [-0.5, 2.0]]$, $\mu = [0, 0]^T$ và $\det(\Sigma) = 0.35$.

Các tham số của phân phối được xác định như sau:

• Vector kỳ vọng: $\mu = [0, 0]^T$.

• Ma trận hiệp phương sai: $\Sigma = [[0.3, -0.5], [-0.5, 2.0]]$.

• Miền đầu vào: $x_1 \in [-2, 2]$, $x_2 \in [-4, 4]$.

1.2. Tính toán giải tích và kiểm định ma trận

Hàm mật độ xác suất chuẩn hai biến được xác định bởi công thức:

1. Định thức ma trận hiệp phương sai:
$\det(\Sigma) = 0.3 \times 2.0 - (-0.5)^2 = 0.35$.
Vì Σ đối xứng, $\det(\Sigma) = 0.35 > 0$ và $\Sigma_{11} = 0.3 > 0$ nên Σ xác định dương theo tiêu chuẩn Sylvester. Đây là điều kiện cần để Σ hợp lệ làm ma trận hiệp phương sai.

2. Ma trận nghịch đảo:
$\Sigma^{-1} = (1/0.35) \times [[2.0, 0.5], [0.5, 0.3]] = [[5.7143, 1.4286], [1.4286, 0.8571]]$.

3. Giá trị cực đại tại tâm phân phối:
$f(0, 0) = 1 / (2\pi\sqrt{0.35}) \approx 0.269021$.
Giá trị này được dùng làm mốc tham chiếu khi quan sát khả năng dự đoán vùng đỉnh của mô hình.

1.3. Đặc điểm hình học của phân phối

Phân phối được chọn có một số đặc điểm đáng chú ý:
• Hiệp phương sai âm: $\Sigma_{12} = \Sigma_{21} = -0.5$. Vì vậy, hai biến có tương quan âm và các đường đồng mức có dạng elip nghiêng.
• Phương sai khác nhau: $\mathrm{Var}(x_1) = 0.3$, $\mathrm{Var}(x_2) = 2.0$. Phương sai theo trục x_2 lớn khoảng 6.67 lần phương sai theo trục x_1; tương ứng, độ lệch chuẩn theo x_2 lớn khoảng 2.58 lần. Mặt cong vì thế trải rộng hơn theo hướng x_2.
• Giá trị ở biên: mật độ giảm khi đi xa tâm nhưng không phải mọi điểm trên biên đều có giá trị nhỏ hơn 10^{-6}. Trên miền đang xét, giá trị lớn nhất dọc biên xấp xỉ 4.93×10^{-3}, tại các điểm gần $(-1, 4)$ và $(1, -4)$. Do đó, khi phân tích sai số cần xét giá trị thực trên toàn miền, không nên mặc định toàn bộ biên bằng 0.

1.4. Định lý Xấp xỉ Phổ quát

Cybenko (1989) chứng minh rằng mạng truyền thẳng có một tầng ẩn, sử dụng hàm kích hoạt sigmoid liên tục phù hợp, có thể xấp xỉ đều các hàm liên tục trên một miền compact. Hornik (1991) đưa ra kết quả tổng quát

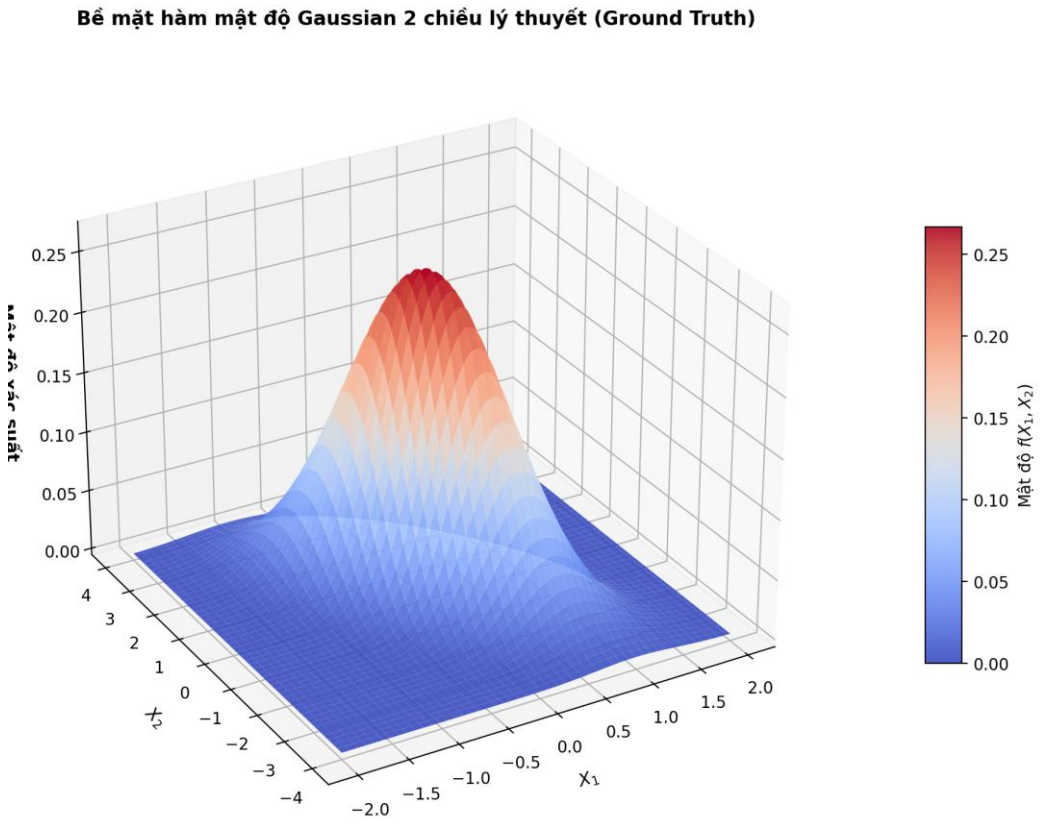
hơn: với hàm kích hoạt liên tục, bị chặn và không hằng, mạng một tầng ẩn cũng có khả năng xấp xỉ các ánh xạ liên tục trên miền compact khi có đủ nơ-ron.

> **Định lý Xấp xỉ Phổ quát**
>
> *Kết quả của Cybenko (1989) xét hàm kích hoạt sigmoid liên tục. Họ hàm một tầng ẩn có dạng:*
>
> $$F(x) = \Sigma_{i=1}^{m} \alpha_i \, \sigma(w_i^T x + b_i)$$
>
> *Với đủ nơ-ron, các hàm thuộc họ này có thể xấp xỉ đều một hàm liên tục trên miền compact với sai số nhỏ tùy ý. Điều kiện không đa thức cho hàm kích hoạt là một kết quả tổng quát hơn, được trình bày trong công trình của Leshno và cộng sự (1993), không nên gán trực tiếp cho định lý Cybenko.*
>
> *Với hàm Tanh được dùng trong báo cáo, có thể viện dẫn kết quả tổng quát của Hornik (1991) về hàm kích hoạt liên tục, bị chặn và không hằng.*

Các định lý này mô tả khả năng biểu diễn về mặt lý thuyết, không xác định trực tiếp số nơ-ron cần dùng, lượng dữ liệu cần có hoặc số epoch để huấn luyện đạt kết quả tốt. Trong đề tài, nhóm sử dụng Tanh; đây là hàm liên tục, bị chặn và không hằng, phù hợp với điều kiện của kết quả tổng quát của Hornik (1991). Các thực nghiệm A, B và C được thiết kế để khảo sát hành vi của mô hình trong một số cấu hình cụ thể, chứ không nhằm chứng minh lại định lý.

1.5. Trực quan hóa mặt cong lý thuyết trong không gian 3D

Nhóm sử dụng module src/visualize.py để tạo lưới tọa độ 100 × 100 trên miền [−2, 2] × [−4, 4], sau đó tính giá trị hàm mật độ tại từng điểm. Hình 1.1 thể hiện mặt cong lý thuyết, làm cơ sở để đối chiếu với kết quả dự đoán của mạng.



Hình 1.1: Mặt cong lý thuyết của hàm mật độ Gaussian 2D

Hình 1.1 cho thấy đỉnh của phân phối nằm tại (0, 0), có giá trị xấp xỉ 0.269. Các đường đồng mức có dạng elip nghiêng, phù hợp với dấu âm của hiệp phương sai; mặt cong mở rộng hơn theo trục x_2 so với trục x_1.

2. THIẾT KẾ VÀ KIỂM ĐỊNH BỘ DỮ LIỆU

2.1. Miền xác định và cách sinh dữ liệu

Dữ liệu đầu vào được lấy mẫu ngẫu nhiên đồng đều trên miền $x_1 \sim U(-2, 2)$ và $x_2 \sim U(-4, 4)$. Với mỗi cặp tọa độ, nhãn y được tính trực tiếp từ công thức Gaussian ở Mục 1.1. Như vậy, dữ liệu là dữ liệu tổng hợp có giá trị mục tiêu được xác định bằng hàm giải tích, không phải dữ liệu đo đạc từ một nguồn bên ngoài.

Để các lần chạy có thể tái lập, nhóm sử dụng bộ sinh số PCG64 của NumPy kết hợp SeedSequence với ROOT_SEED = 20261024. Các tập Train Full, Validation và Test lần lượt dùng spawn_key 0, 1 và 2. Việc tách chuỗi sinh số giúp các tập được tạo độc lập, đồng thời hạn chế việc thay đổi quy mô một tập làm ảnh hưởng đến dữ liệu của tập còn lại.

2.2. Kiểm tra tính độc lập và toàn vẹn của dữ liệu

Bộ dữ liệu gồm 2.500 mẫu và được chia thành ba tập như trình bày trong Bảng 2.1:

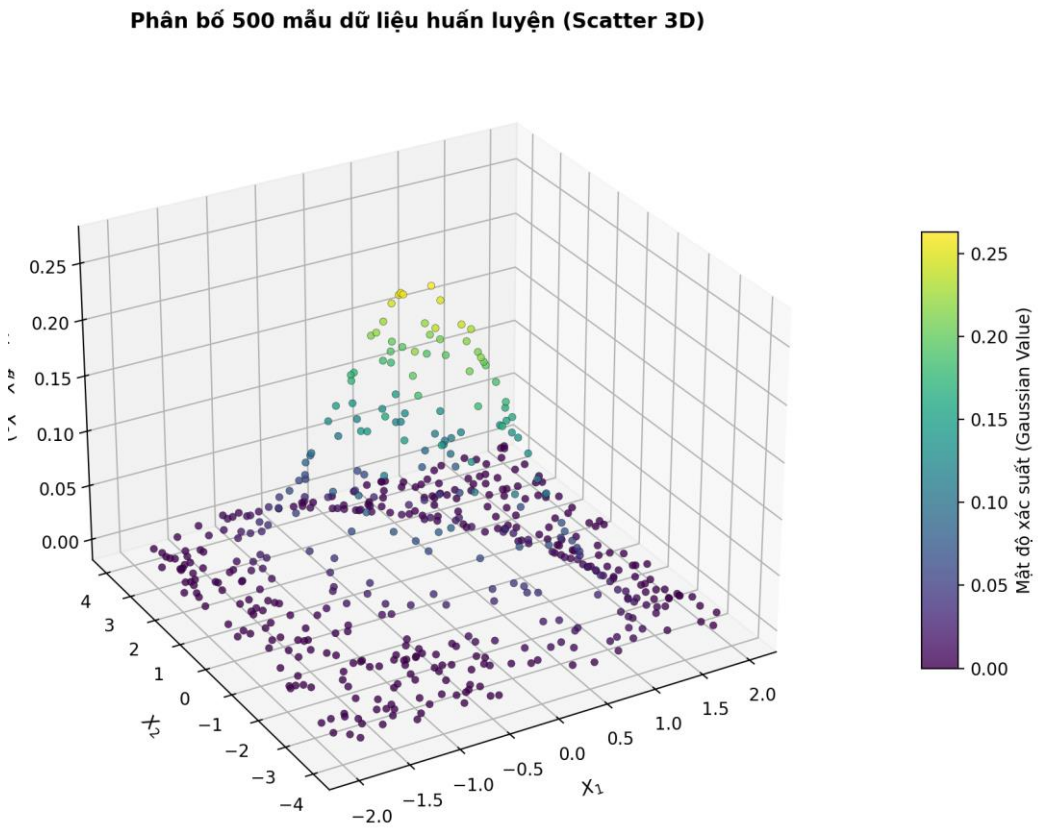
Tập dữ liệu	Số lượng mẫu	Tỷ lệ (%)	Mục đích sử dụng
Train Full (train_full.csv)	2.000	80.0%	Huấn luyện mô hình cơ sở và tập nguồn trích chọn mẫu cho Thực nghiệm A
Validation (val.csv)	400	16.0%	Theo dõi quá trình huấn luyện, lưu Checkpoint tối ưu (best_model.pt)
Test độc lập (test.csv)	100	4.0%	Đánh giá năng lực tổng quát hóa cuối cùng, độc lập hoàn toàn với quá trình train

Bảng 2.1: Quy mô và cách phân chia các tập dữ liệu thực nghiệm

Nhóm dùng các kiểm tra trong src/data.py để rà soát dữ liệu trước khi huấn luyện:
1. Không có điểm dữ liệu trùng nhau giữa các cặp Train/Validation, Train/Test và Validation/Test.
2. Không ghi nhận giá trị NaN, Null hoặc Inf trong 2.500 mẫu.
3. Giá trị y tính từ công thức giải tích được đối chiếu với phân phối MultivariateNormal của PyTorch; sai số tương đối lớn nhất được ghi nhận nhỏ hơn 10^{-12} khi tính bằng float64.

2.3. Trực quan hóa phân bố mẫu huấn luyện

Hình 2.1 hiển thị 500 điểm được lấy từ tập train_full.csv, vốn có tổng cộng 2.000 mẫu huấn luyện. Trục đứng và màu sắc biểu thị giá trị mật độ xác suất tại từng cặp tọa độ.



Hình 2.1: Phân bố 3D của 500 điểm hiển thị từ tập train_full gồm 2.000 mẫu

2.4. Chuẩn hóa đặc trưng và kiểm soát rò rỉ dữ liệu

Hai đặc trưng đầu vào có khoảng giá trị khác nhau: x_1 thuộc $[-2, 2]$, còn x_2 thuộc $[-4, 4]$. Nhóm chuẩn hóa từng đặc trưng theo phương pháp Z-score để đưa chúng về thang đo tương đồng trước khi đưa vào mạng.

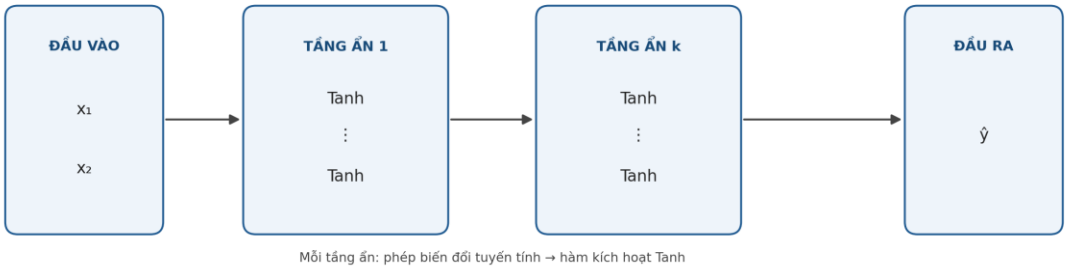
$\tilde{x}_j = (x_j - \mu_j) / \sigma_j$, trong đó μ_j và σ_j lần lượt là trung bình và độ lệch chuẩn của đặc trưng thứ j, được tính trên tập huấn luyện.

> **Nguyên tắc chuẩn hóa dữ liệu**
> *Scaler chỉ được khớp tham số trên X_train. Sau đó, chính các tham số này được dùng để biến đổi X_val và X_test. Không tính lại trung bình hoặc độ lệch chuẩn trên hai tập sau, nhằm tránh đưa thông tin từ tập đánh giá vào quá trình tiền xử lý.*

3. XÂY DỰNG MÔ HÌNH MLP VÀ QUY TRÌNH HUẤN LUYỆN

3.1. Thiết kế kiến trúc mạng MLPRegressor trong PyTorch

Nhóm xây dựng lớp MLPRegressor kế thừa nn.Module trong src/model.py. Kiến trúc được tạo linh hoạt theo danh sách số nơ-ron ở các tầng ẩn, nhờ đó có thể dùng chung cho thực nghiệm độ rộng và độ sâu.



Hình 3.1: Sơ đồ kiến trúc tổng quát của mạng MLPRegressor

- Tầng đầu vào nhận hai đặc trưng x_1 và x_2 (input_dim = 2).
- Các tầng ẩn được khai báo theo danh sách hidden_layers = [h_1, h_2, …, h_k]. Mỗi tầng gồm một lớp nn.Linear, sau đó là hàm kích hoạt Tanh.
- Tầng đầu ra dùng nn.Linear(h_k, 1) và không gắn hàm kích hoạt. Cách thiết kế này phù hợp với bài toán hồi quy vì đầu ra là một giá trị thực. Sigmoid giới hạn đầu ra trong (0, 1); Softplus tạo đầu ra dương. Tuy nhiên, đề tài không đặt ràng buộc đầu ra bằng một hàm kích hoạt, mà để mô hình học trực tiếp giá trị cần dự đoán.

3.2. Lựa chọn hàm kích hoạt Tanh

Trong các thực nghiệm, nhóm sử dụng Tanh ở các tầng ẩn. Lựa chọn này phù hợp với mục tiêu xấp xỉ một hàm trơn và có đầu ra đối xứng quanh 0 của hàm kích hoạt, đồng thời Tanh có đạo hàm liên tục.

Hàm Tanh được định nghĩa bởi $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$, với đạo hàm $\tanh'(z) = 1 - \tanh^2(z)$. Giá trị đạo hàm nằm trong (0, 1], và tiến gần 0 khi |z| lớn. Vì vậy, Tanh có thể gặp bão hòa ở các vùng đầu vào lớn về trị tuyệt đối.

ReLU(z) = max(0, z) là hàm tuyến tính từng khúc, nên mỗi vùng mà trạng thái kích hoạt không đổi sẽ tạo ra một ánh xạ tuyến tính. Điều này không có nghĩa mạng ReLU không thể xấp xỉ hàm trơn: mạng ReLU vẫn có khả năng xấp xỉ hàm liên tục với độ chính xác tùy ý khi đáp ứng các điều kiện thích hợp. Tanh được chọn trong đề tài vì có dạng trơn; tuy nhiên, nhóm không thực hiện thí nghiệm đối đầu Tanh và ReLU nên chưa thể kết luận Tanh luôn cho kết quả tốt hơn.

3.3. Tầng đầu ra tuyến tính và hàm mất mát MSE

Vì đây là bài toán hồi quy, nhóm sử dụng sai số bình phương trung bình (MSE) làm hàm mất mát:

$$L_MSE = (1/N) \Sigma_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Trong đó y_i là giá trị lý thuyết, $\hat{y}_i$ là giá trị mô hình dự đoán và N là số mẫu. MSE bình phương sai số nên những dự đoán lệch nhiều đóng góp lớn hơn vào tổng mất mát; hàm này không tự động chỉ tập trung vào vùng đỉnh mà đánh giá sai số trên tất cả mẫu trong batch.

3.4. Thuật toán tối ưu Adam và cấu hình huấn luyện cơ sở

Nhóm sử dụng Adam với learning_rate = 0.001. Adam điều chỉnh tốc độ cập nhật dựa trên các ước lượng moment bậc nhất và bậc hai của gradient. Cấu hình được giữ thống nhất giữa các lần chạy để việc so sánh tập trung vào biến số đang khảo sát.

Các siêu tham số dùng chung được liệt kê trong Bảng 3.1:

Siêu tham số	Giá trị cấu hình cơ sở
Kích thước lô	32
Thuật toán tối ưu	Adam (lr = 0.001, betas = (0.9, 0.999))
Hàm kích hoạt các tầng ẩn	Tanh
Số Epoch huấn luyện tối đa	300 epoch
Kiến trúc mạng cơ sở	2 tầng ẩn [16, 16] – 337 tham số
Số lần lặp ngẫu nhiên đánh giá	3 seed: [0, 1, 2]

Bảng 3.1: Cấu hình huấn luyện cơ sở của mô hình

3.5. Huấn luyện theo mini-batch và lưu mô hình tốt nhất

Quy trình trong src/trainer.py gồm các bước:

1. DataLoader chia dữ liệu huấn luyện thành các mini-batch có batch_size = 32 và bật shuffle.
2. Ở mỗi batch, mô hình thực hiện lan truyền thuận, tính MSE, gọi backward() để tính gradient và optimizer.step() để cập nhật trọng số.
3. Sau mỗi epoch, mô hình được chuyển sang chế độ đánh giá và tính loss trên 400 mẫu Validation bằng torch.no_grad().
4. Nhóm lưu state_dict tại epoch có validation loss thấp nhất vào checkpoints/best_model.pt. Trước khi đánh giá trên tập Test, mô hình được khôi phục từ checkpoint này.

4. THỰC NGHIỆM A — ẢNH HƯỞNG CỦA QUY MÔ DỮ LIỆU HUẤN LUYỆN

4.1. Thiết kế thực nghiệm và cách đánh giá

Thực nghiệm A khảo sát mối liên hệ giữa số lượng mẫu huấn luyện và sai số của mô hình MLP. Biến được thay đổi là số mẫu A; các yếu tố còn lại được giữ cố định theo cấu hình cơ sở.

- Số mẫu huấn luyện A nhận 10 giá trị: 200, 400, 600, 800, 1.000, 1.200, 1.400, 1.600, 1.800 và 2.000.
- Để các tập con có tính lồng nhau và thuận tiện so sánh, nhóm lấy A dòng đầu tiên từ train_full.csv ở mỗi mốc.
- Các yếu tố giữ cố định gồm kiến trúc [16, 16], Tanh, Adam với learning rate 0.001, batch size 32 và tối đa 300 epoch. Tập Validation gồm 400 mẫu và tập Test gồm 100 mẫu, không thay đổi giữa các mốc.
- Mỗi mốc A được chạy với ba seed 0, 1 và 2. Bảng báo cáo giá trị trung bình và độ lệch chuẩn của các chỉ số.

4.2. Kết quả thực nghiệm A

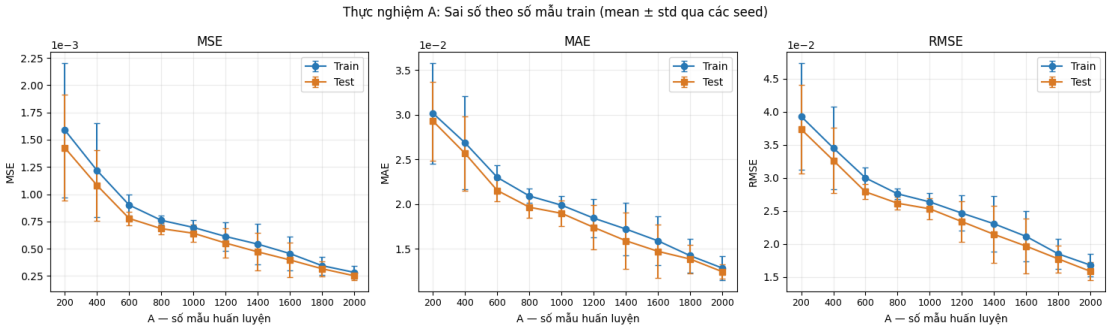
Bảng 4.1 trình bày MSE trên tập huấn luyện, MSE, MAE và RMSE trên tập Test theo từng quy mô dữ liệu:

Số mẫu (A)	MSE huấn luyện (TB ± ĐLC)	MSE kiểm thử (TB ± ĐLC)	MAE kiểm thử (TB ± ĐLC)	RMSE kiểm thử (TB ± ĐLC)
200	0.001587 ± 0.000615	0.001425 ± 0.000486	0.029282 ± 0.004385	0.037344 ± 0.006734
400	0.001217 ± 0.000431	0.001081 ± 0.000322	0.025659 ± 0.004138	0.032631 ± 0.004943
600	0.000900 ± 0.000095	0.000777 ± 0.000063	0.021530 ± 0.001206	0.027868 ± 0.001144
800	0.000760 ± 0.000042	0.000684 ± 0.000053	0.019660 ± 0.001180	0.026141 ± 0.001009
1.000	0.000695 ± 0.000071	0.000642 ± 0.000081	0.018979 ± 0.001411	0.025302 ± 0.001624
1.200	0.000612 ± 0.000129	0.000552 ± 0.000136	0.017410 ± 0.002457	0.023361 ± 0.003030
1.400	0.000542 ± 0.000185	0.000471 ± 0.000173	0.015915 ± 0.003172	0.021430 ± 0.004271
1.600	0.000456 ± 0.000156	0.000397 ± 0.000160	0.014731 ± 0.002983	0.019641 ± 0.004161
1.800	0.000345 ± 0.000083	0.000316 ± 0.000070	0.013870 ± 0.001587	0.017712 ± 0.002030
2.000	0.000283 ± 0.000057	**0.000252 ± 0.000042**	0.012459 ± 0.000816	0.015831 ± 0.001372

Bảng 4.1: Kết quả thực nghiệm A theo số lượng mẫu huấn luyện

4.3. Đường cong sai số theo quy mô dữ liệu

Hình 4.1 biểu diễn sự thay đổi của MSE, MAE và RMSE khi tăng số mẫu huấn luyện. Các thanh sai số thể hiện độ lệch chuẩn qua ba lần chạy.



Hình 4.1: Sai số theo số lượng mẫu huấn luyện

4.4. Nhận xét về ảnh hưởng của số lượng mẫu

Trong phạm vi cấu hình và dữ liệu đã thử, Test MSE trung bình giảm từ 0.001425 tại A = 200 xuống 0.000252 tại A = 2.000; Test MAE giảm từ 0.029282 xuống 0.012459. Giá trị Test MSE giảm ở cả 9 bước tăng quy mô liên tiếp.

Mức giảm từ 200 lên 800 mẫu là 0.000741, tương đương khoảng 52% so với mốc 200 mẫu. Từ 1.000 lên 2.000 mẫu, mức giảm là 0.000390 dù số mẫu tăng thêm nhiều hơn. Điều này cho thấy lợi ích trung bình trên mỗi mẫu bổ sung có xu hướng thấp hơn ở vùng dữ liệu lớn.

Kết quả này chỉ phản ánh các lần chạy trong thiết kế thực nghiệm A. Chưa có kiểm định riêng để xác định

nguyên nhân chính của xu hướng trên, vì vậy nhóm không kết luận rằng mọi bộ dữ liệu hoặc mọi kiến trúc đều có cùng quy luật.

5. THỰC NGHIỆM B — ẢNH HƯỞNG CỦA ĐỘ RỘNG TẦNG ẨN

5.1. Thiết kế thực nghiệm với một tầng ẩn

Thực nghiệm B khảo sát khả năng biểu diễn của mạng một tầng ẩn khi thay đổi số nơ-ron. Kết quả được đối chiếu với nhận định của định lý xấp xỉ phổ quát, nhưng không được xem là phép chứng minh định lý.

- Kiến trúc: 2 → B → 1, sử dụng Tanh ở tầng ẩn và đầu ra tuyến tính.
- Độ rộng B gồm 13 mốc: 1 đến 10, sau đó là 16, 32 và 64 nơ-ron.
- Tập huấn luyện gồm 2.000 mẫu; batch size = 32, learning rate = 0.001, tối đa 300 epoch và ba seed được giữ thống nhất.

5.2. Kết quả thực nghiệm B

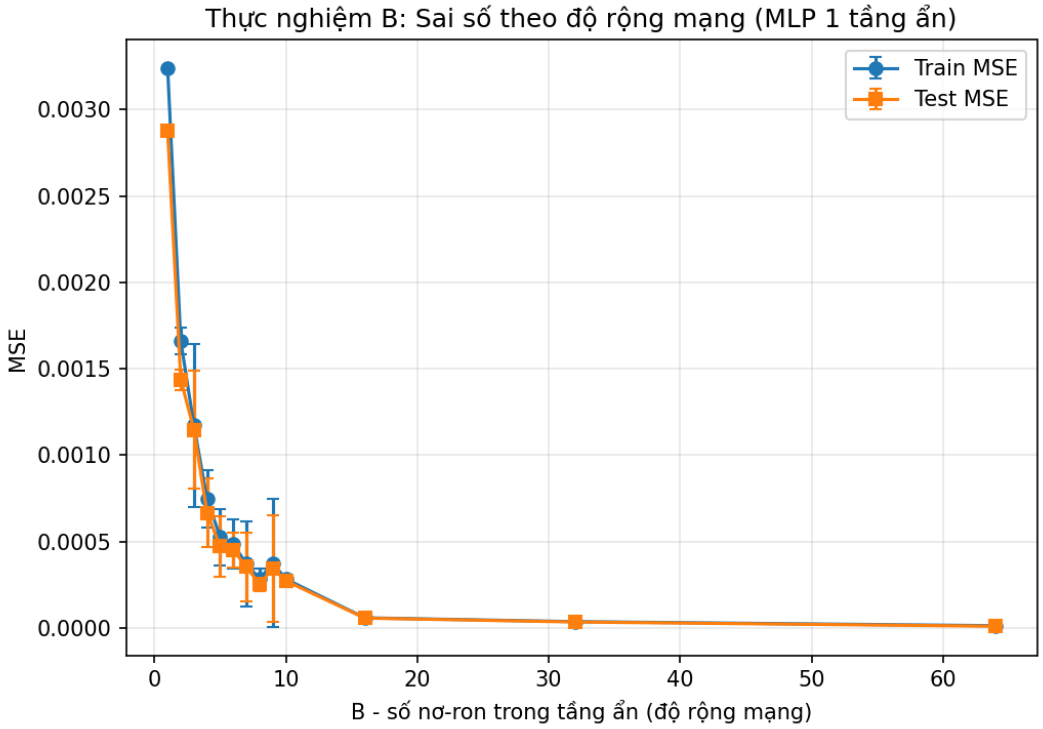
Bảng 5.1 tổng hợp số tham số, sai số và thời gian huấn luyện ở 13 mốc độ rộng:

Độ rộng (B)	Số tham số	MSE huấn luyện (TB)	MSE kiểm thử (TB ± ĐLC)	MAE kiểm thử (TB)	Thời gian (giây)
1	5	0.003242	0.002875 ± 0.000003	0.038337	33.2
2	9	0.001662	0.001437 ± 0.000058	0.022746	33.5
3	13	0.001173	0.001147 ± 0.000341	0.023213	33.6
4	17	0.000747	0.000666 ± 0.000201	0.018472	33.8
5	21	0.000525	0.000472 ± 0.000175	0.016318	34.1
6	25	0.000483	0.000449 ± 0.000100	0.016183	34.2
7	29	0.000370	0.000353 ± 0.000200	0.013879	34.5
8	33	0.000283	0.000252 ± 0.000042	0.012459	34.8
9	37	0.000374	0.000345 ± 0.000308	0.012791	35.0
10	41	0.000282	0.000271 ± 0.000030	0.012210	35.3
16	65	0.000057	0.000056 ± 0.000002	0.005780	36.2
32	129	0.000035	0.000033 ± 0.000014	0.004492	38.5
64	257	0.000012	**0.000009 ± 0.000001**	0.002386	42.1

Bảng 5.1: Kết quả thực nghiệm B theo độ rộng tầng ẩn

5.3. Nhận xét về độ rộng tầng ẩn

Hình 5.1 thể hiện sự thay đổi của sai số theo độ rộng B. Hình 5.2 đối chiếu giá trị dự đoán với giá trị thực trên tập kiểm thử ở cấu hình B = 64.



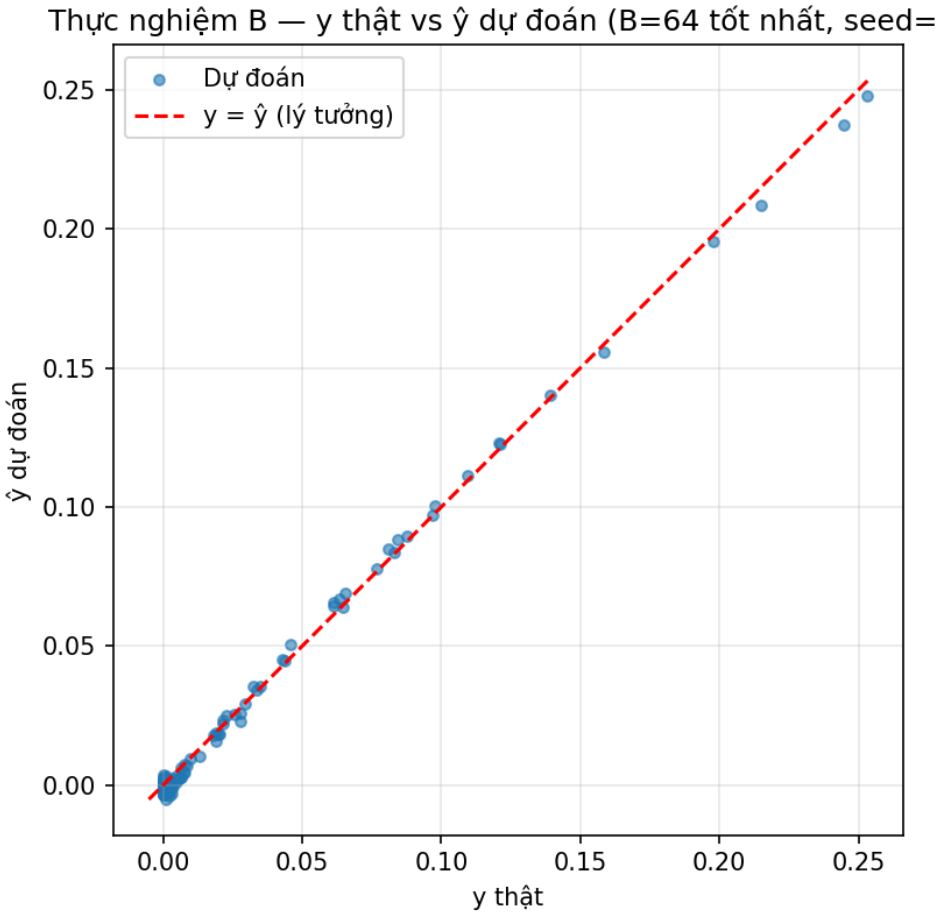
Hình 5.1: Sai số theo độ rộng tầng ẩn

Ở các cấu hình B = 1, 2 và 3, số tham số của mạng lần lượt chỉ là 5, 9 và 13; Test MSE còn tương đối cao. Đây là dấu hiệu mô hình chưa đủ khả năng biểu diễn mục tiêu trong cấu hình đang xét.

Khi tăng lên B = 4, Test MSE giảm từ 0.002875 tại B = 1 xuống 0.000666. Sau đó, sai số tiếp tục giảm ở các mốc lớn hơn. Tại B = 64, mạng có 257 tham số và đạt Test MSE trung bình 8.93×10^{-6}, thấp nhất trong 13 cấu hình đã thử.

Xu hướng này phù hợp với nhận định rằng mạng một tầng ẩn có thể cải thiện khả năng xấp xỉ khi tăng số nơ-ron. Tuy nhiên, một số lượng hữu hạn cấu hình thực nghiệm không chứng minh được tính xấp xỉ với sai số tùy ý của định lý.

5.4. Số tham số và thời gian huấn luyện



Hình 5.2: Đối chiếu giá trị thực và giá trị dự đoán với B = 64

Với mạng một tầng ẩn có B nơ-ron, số tham số là $P(B) = (2B + B) + (B + 1) = 4B + 1$. Khi B tăng từ 1 lên 64, thời gian huấn luyện ghi nhận tăng từ 33.2 giây lên 42.1 giây, tương đương khoảng 27%. Trong tập cấu hình đã thử, $B = 64$ cho sai số thấp nhất nhưng cũng cần nhiều tham số hơn. Đây là cơ sở để tiếp tục khảo sát việc phân bổ nơ-ron qua nhiều tầng ở thực nghiệm C.

6. THỰC NGHIỆM C — ẢNH HƯỞNG CỦA ĐỘ SÂU MẠNG VÀ TRỰC QUAN HÓA 3D

6.1. Thiết kế thực nghiệm với nhiều tầng ẩn

Thực nghiệm C thay đổi số tầng ẩn, đồng thời giữ cố định độ rộng ở mỗi tầng để xem độ sâu ảnh hưởng như thế nào đến sai số và thời gian huấn luyện.

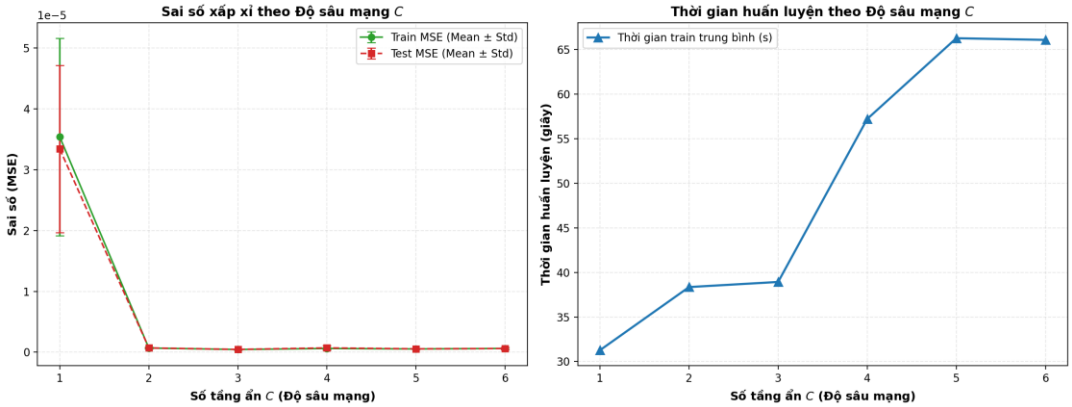
- Số tầng ẩn C nhận các giá trị 1, 2, 3, 4, 5 và 6.
- Mỗi tầng có 32 nơ-ron. Các cấu hình lần lượt là [32], [32, 32], [32, 32, 32] và tiếp tục đến sáu tầng, mỗi tầng 32 nơ-ron.
- Các yếu tố còn lại được giữ cố định: 2.000 mẫu huấn luyện, batch size = 32, learning rate = 0.001, Adam, tối đa 300 epoch và ba seed.

6.2. Kết quả theo độ sâu mạng

Bảng 6.1 tổng hợp số tham số, sai số trên tập Test, hệ số xác định R^2 và thời gian huấn luyện:

Độ sâu (C)	Số tham số	MSE kiểm thử (TB ± ĐLC)	MAE kiểm thử (TB)	Hệ số R^2 kiểm thử	Thời gian (giây)
1	129	3.34e-05 ± 1.38e-05	0.004492	0.9859	31.3
2	1.185	7.06e-07 ± 9.38e-08	0.000624	0.9997	38.4
3 (Tối ưu)	**2.241**	**4.56e-07 ± 7.04e-08**	**0.000511**	**0.9998**	**38.9**
4	3.297	6.08e-07 ± 7.97e-08	0.000572	0.9997	47.7
5	4.353	6.61e-07 ± 1.54e-07	0.000599	0.9997	57.3
6	5.409	6.26e-07 ± 1.28e-07	0.000588	0.9997	66.0

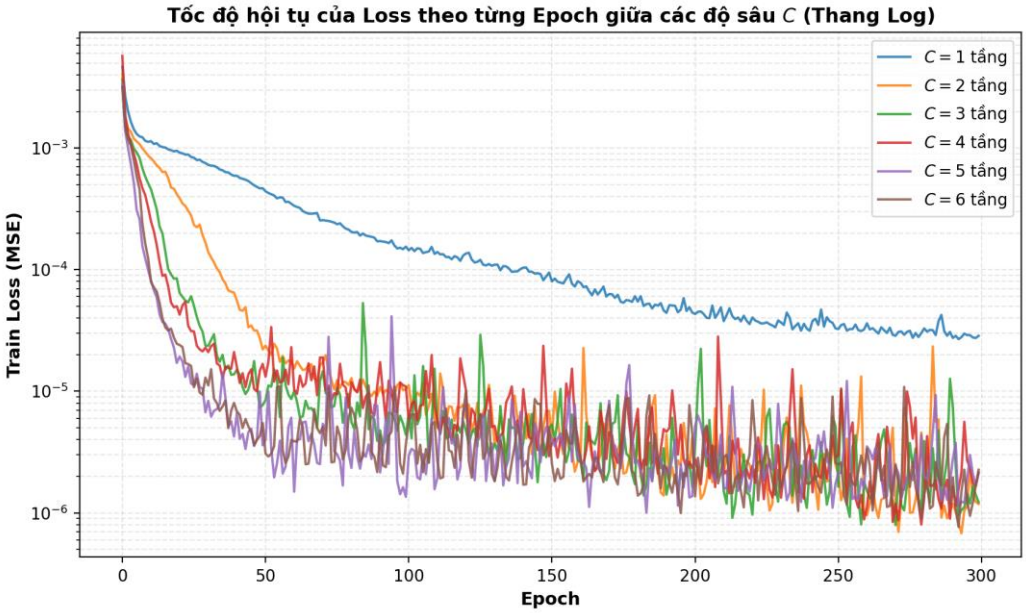
Bảng 6.1: Kết quả thực nghiệm C theo số tầng ẩn



Hình 6.1: Sai số kiểm thử và thời gian huấn luyện theo độ sâu mạng

6.3. Quá trình thay đổi hàm mất mát

Hình 6.2 thể hiện sự thay đổi của hàm mất mát theo epoch trên thang logarit. Đồ thị giúp quan sát tốc độ giảm loss và mức dao động trong quá trình huấn luyện.



Hình 6.2: Đường cong hàm mất mát theo epoch

6.4. So sánh kết quả theo độ sâu và bàn luận về tiêu biến gradient

Trong sáu cấu hình được khảo sát, C = 1 có Test MSE trung bình 3.34×10^{-5}. Khi tăng lên C = 2, chỉ số này giảm xuống 7.06×10^{-7}. Cấu hình C = 3, gồm ba tầng ẩn với 32 nơ-ron mỗi tầng, đạt Test MSE thấp nhất: $4.56 \times 10^{-7} \pm 7.04 \times 10^{-8}$; Test MAE là 0.000511 và R^2 là 0.9998.

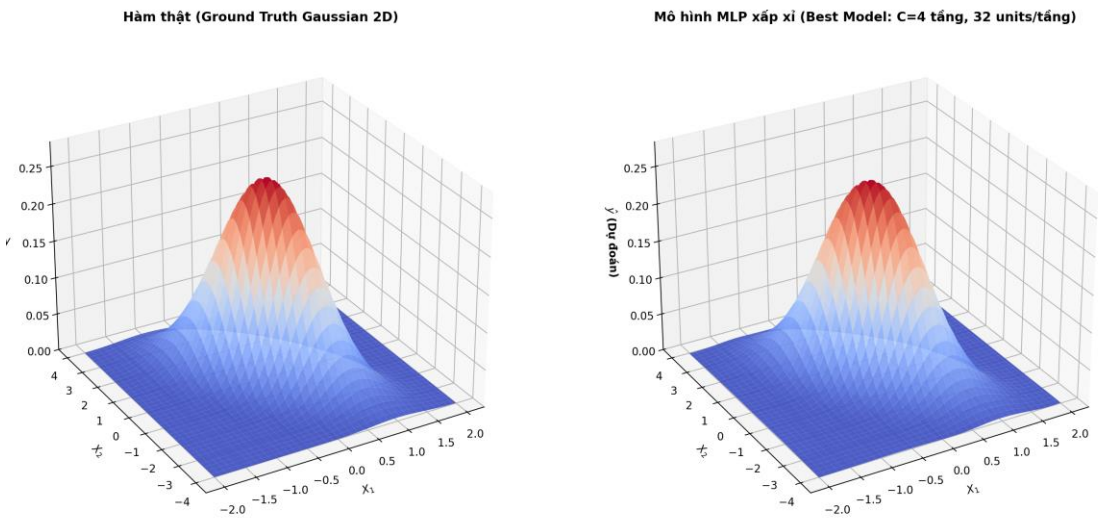
Khi tăng độ sâu lên 4, 5 hoặc 6 tầng, Test MSE không tiếp tục giảm mà dao động trong khoảng 6.08×10^{-7} đến 6.61×10^{-7}. Trong khi đó, thời gian huấn luyện tăng từ 31.3 giây ở C = 1 lên 66.0 giây ở C = 6. Vì vậy, C = 3 là cấu hình có kết quả tốt nhất trong phạm vi thực nghiệm này, không phải kết luận về tối ưu toàn cục cho mọi thiết kế mạng.

Tiêu biến gradient là một khả năng có thể xem xét khi mạng sâu hơn: đạo hàm Tanh tiến gần 0 khi đầu vào có trị tuyệt đối lớn, và phép nhân liên tiếp các đạo hàm trong lan truyền ngược có thể làm gradient suy giảm. Tuy nhiên, nhóm chưa đo norm gradient theo từng tầng, nên dữ liệu Test MSE và thời gian huấn luyện hiện tại chưa đủ để khẳng định đây là nguyên nhân trực tiếp của kết quả ở C = 4–6.

> **Lưu ý khi diễn giải hiện tượng tiêu biến gradient**
>
> *Đạo hàm của Tanh là $\tanh'(z) = 1 - \tanh^2(z)$, có giá trị trong (0, 1]. Khi $|z|$ lớn, đạo hàm có thể rất nhỏ. Trong quá trình lan truyền ngược, các đạo hàm được nhân liên tiếp qua các tầng; nếu nhiều tầng ở vùng bão hòa, gradient truyền về các tầng đầu có thể suy giảm.*
>
> *Đây là giải thích lý thuyết cho một khả năng có thể xảy ra. Báo cáo hiện chưa có số đo norm gradient theo từng tầng, vì vậy không xem tiêu biến gradient là nguyên nhân đã được xác nhận cho kết quả của thực nghiệm C.*

6.5. So sánh mặt cong dự đoán với mặt cong lý thuyết

Tiêu đề được ghi trực tiếp trên Hình 6.3 cho biết đồ thị đang hiển thị mô hình có 4 tầng ẩn, mỗi tầng 32 nơ-ron. Vì vậy, hình này minh họa bề mặt dự đoán của cấu hình C = 4, không phải cấu hình C = 3 có Test MSE thấp nhất trong Bảng 6.1. Muốn đối chiếu trực quan đúng với cấu hình có sai số thấp nhất, cần xuất lại hình từ checkpoint C = 3.

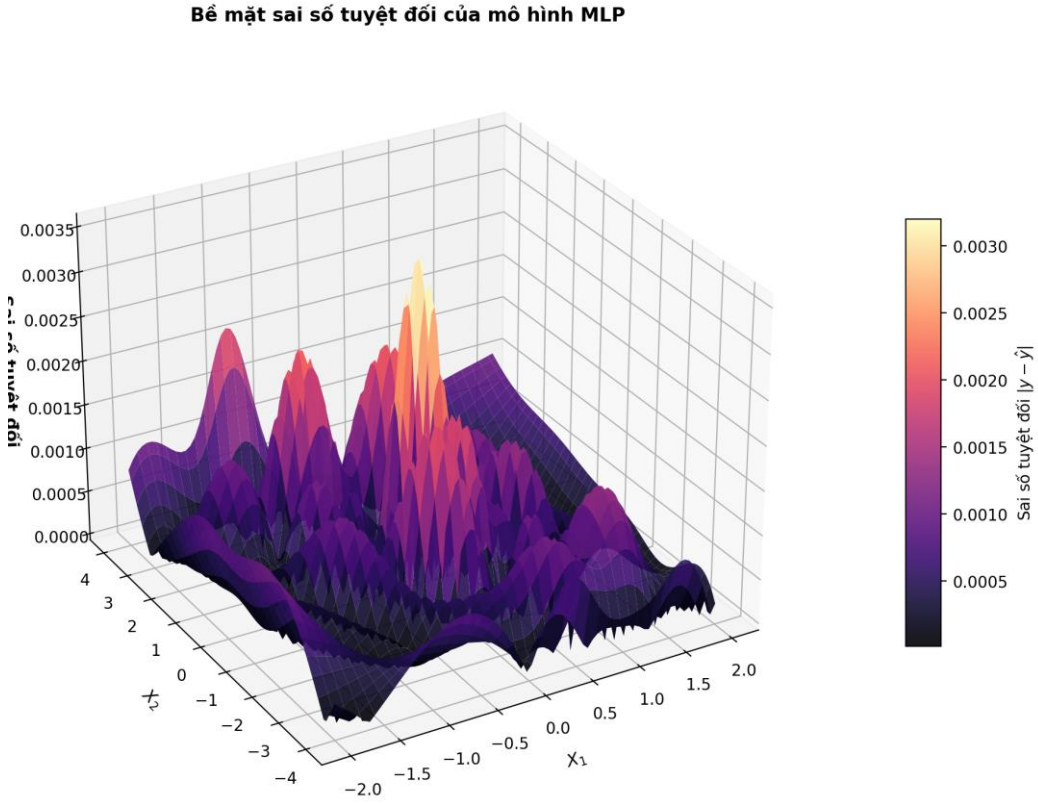


Hình 6.3: Minh họa mặt cong dự đoán của mô hình C = 4 theo nhãn trên hình

Quan sát Hình 6.3 có thể thấy mô hình C = 4 đã tái hiện được dạng đỉnh và xu hướng nghiêng của phân phối. Tuy nhiên, đây là nhận xét về hình minh họa C = 4; kết luận cấu hình có Test MSE thấp nhất vẫn dựa trên Bảng 6.1, trong đó C = 3 có sai số thấp nhất.

6.6. Phân tích bề mặt sai số

Nhóm tính sai số tuyệt đối tại từng điểm trên lưới 100 × 100 theo công thức $e(x_1, x_2) = |y - \hat{y}|$. Hình 6.4 biểu diễn bề mặt sai số trên toàn miền xác định.



Hình 6.4: Bề mặt sai số tuyệt đối $|y - \hat{y}|$ trên miền khảo sát

Hình 6.4 cho thấy sai số không phân bố đồng đều trên toàn miền mà tập trung thành các vùng cục bộ. Thang màu của đồ thị đạt đến khoảng 3.5×10^{-3}. Các vị trí có sai số lớn cần được xem xét cùng độ dốc và độ cong của hàm mục tiêu. Do báo cáo chưa cung cấp thống kê tỷ lệ diện tích theo từng ngưỡng sai số, nhóm không đưa ra nhận định định lượng rằng hơn 95% diện tích có sai số bằng 0.

7. TỔNG KẾT, THẢO LUẬN VÀ KẾT LUẬN

7.1. Tổng hợp ba trục thực nghiệm A, B và C

Bảng 7.1 tóm tắt cấu hình có Test MSE thấp nhất được ghi nhận trong từng thực nghiệm:

Thực nghiệm	Phạm vi khảo sát	Cấu hình có sai số thấp nhất	Test MSE thấp nhất
Thực nghiệm A (Quy mô)	A = 200–2.000 mẫu (10 mốc)	A = 2.000 mẫu	0.000252 ± 0.000042 (giảm khoảng 6 lần so với A = 200)
Thực nghiệm B (Độ rộng)	B = 1–64 nơ-ron (13 mốc, 1 tầng ẩn)	B = 64 nơ-ron (257 tham số)	$8.93 \times 10^{-6} \pm 1.19 \times 10^{-6}$
Thực nghiệm C (Độ sâu)	C = 1–6 tầng ẩn (32 nơ-ron/tầng)	C = 3 tầng ẩn [32, 32, 32] (2.241 tham số)	$4.56 \times 10^{-7} \pm 7.04 \times 10^{-8}$ (thấp nhất trong thực nghiệm C)

Bảng 7.1: Tổng hợp kết quả ba thực nghiệm A, B và C

7.2. Cấu hình đề xuất cho bài toán

Từ các kết quả đã ghi nhận, nhóm đề xuất dùng cấu hình sau làm điểm xuất phát cho bài toán xấp xỉ Gaussian 2D:

• Dữ liệu: 2.000 mẫu huấn luyện; thực nghiệm A cho thấy sai số giảm khi tăng quy mô từ 200 đến 2.000 mẫu.

• Kiến trúc: ba tầng ẩn [32, 32, 32], hàm kích hoạt Tanh và tầng đầu ra tuyến tính; cấu hình này đạt Test MSE $4.56 \times 10^{-7} \pm 7.04 \times 10^{-8}$ trong thực nghiệm C.

• Huấn luyện: Adam, learning rate = 0.001, batch size = 32, tối đa 300 epoch và lưu checkpoint theo validation loss.

Cần lưu ý rằng thực nghiệm A dùng kiến trúc cơ sở [16, 16], còn thực nghiệm C dùng 2.000 mẫu. Vì nhóm chưa chạy một thí nghiệm kết hợp riêng để thay đổi đồng thời số mẫu và kiến trúc [32, 32, 32], cấu hình trên là đề xuất tổng hợp từ các kết quả riêng lẻ, chưa được xác nhận là tối ưu toàn diện trong một lần chạy độc lập.

7.3. Khả năng tái lập và hướng dẫn chạy mã nguồn

Dự án sử dụng đường dẫn tương đối thông qua pathlib.Path(__file__).resolve().parent.parent để xác định thư mục gốc, giúp mã nguồn thuận tiện hơn khi chạy trên các hệ điều hành khác nhau. Để tái hiện thí nghiệm, người dùng cần cài đặt các thư viện theo requirements.txt và chạy các lệnh được liệt kê bên dưới. Kết quả thời gian chạy có thể thay đổi theo phần cứng và môi trường phần mềm.

> **Hướng dẫn thực thi độc lập trên từng máy:**
> *# Cài đặt môi trường:*
> *pip install -r requirements.txt*
>
> *# Chạy kiểm tra dữ liệu và trực quan hóa 3D:*
> *python src/visualize.py*
>
> *# Chạy độc lập từng thực nghiệm:*
> *python experiments/exp_A_samples.py*
> *python experiments/exp_B_width.py*
> *python experiments/exp_C_depth.py*

7.4. Kết luận và kinh nghiệm thực hiện

Qua đề tài, nhóm đã thực hiện được các nội dung chính sau:

1. Xây dựng hàm Gaussian 2D từ các tham số của phân phối chuẩn hai biến và dùng hàm này tạo dữ liệu cho bài toán hồi quy.
2. Cài đặt mạng MLP bằng PyTorch, chuẩn hóa dữ liệu, huấn luyện theo mini-batch và lưu mô hình dựa trên validation loss.
3. Khảo sát ảnh hưởng của quy mô dữ liệu, độ rộng và độ sâu mạng. Trong phạm vi đã thử, số mẫu lớn hơn và số nơ-ron nhiều hơn thường giúp giảm sai số; cấu hình ba tầng ẩn cho kết quả tốt nhất trong thực nghiệm C.
4. Thực hành quản lý mã nguồn, tổng hợp kết quả và trực quan hóa mặt cong dự đoán cùng sai số.

Kết quả cũng cho thấy việc giải thích một cơ chế như tiêu biến gradient cần có số đo phù hợp, chẳng hạn norm gradient theo tầng, thay vì chỉ suy luận từ sai số kiểm thử.

7.5. Tài liệu tham khảo và thông tin bàn giao

- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals, and Systems, 2(4), 303–314. https://doi.org/10.1007/BF02551274
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359–366. https://doi.org/10.1016/0893-6080(89)90020-8
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. Neural Networks, 4(2), 251–257. https://doi.org/10.1016/0893-6080(91)90009-T
- Leshno, M., Lin, V. Y., Pinkus, A., & Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. Neural Networks, 6(6), 861–867. https://doi.org/10.1016/S0893-6080(05)80131-5
- Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Kho lưu trữ mã nguồn của nhóm: *https://github.com/SGHPK0905/DeepLearning_3SV_DDU1231*